

SC2001 Lab 1 : MergeInsert

SDDB Group 6

- Wang Yongqing
- Johannes Yap
- Ong Jee Shen
- Balodi Shalok

Merge Sort

```
9 + void merge(vector<int>& arr, int left, int mid, int right){
10 +     int n1 = mid-left+1;
11 +     int n2 = right-mid;
12 +
13 +     // create temporary vectors by left and copying over
14 +     vector<int> l(n1), r(n2);
15 +     for(int i=left; i<mid; i++){
16 +         l[i] = arr[left+i];
17 +     }
18 +     for(int j=mid; j<right; j++){
19 +         r[j] = arr[mid+j];
20 +     }
21 +
22 +     // merge arrays back by comparing each of left and right
23 +     int i=0, j=0;
24 +     int k = left;
25 +     while (i<n1 && j<n2){
26 +         if(l[i]<=r[j]){
27 +             arr[k] = l[i];
28 +             i++;
29 +         }
30 +         else{
31 +             arr[k] = r[j];
32 +             j++;
33 +         }
34 +         k++;
35 +     }
36 +
37 +     // add remaining elements of l and r into the array
38 +     while(i<n1){
39 +         arr[k] = l[i];
40 +         k++;
41 +         i++;
42 +     }
43 +     while(j<n2){
44 +         arr[k] = r[j];
45 +         k++;
46 +         j++;
47 +     }
48 + }
```

Insertion Sort

```
49 + void insert(vector<int>& arr, int left, int right){
50 +     // assume the first index is already sorted
51 +     for (int i=left+1; i<=right; i++){
52 +         int key = arr[i];
53 +         int j=i-1;
54 +         // keep swapping until key > j
55 +         while (j>= left && key < arr[j]){
56 +             arr[j+1] = arr[j];
57 +             j--;
58 +         }
59 +         arr[j+1] = key;
60 +     }
61 + }
```

Hybrid Algorithm (Merge + Insert Sort)

```
62 + void hybridSort(vector<int>& arr, int left, int right){
63 +     if(left >= right)
64 +         return;
65 +     int diff = right-left+1;
66 +     if (diff<=5){
67 +         insert(arr,left, right);
68 +         return;
69 +     }
70 +     // recursively split the arrays first
71 +     int mid = left + (right - left) / 2;
72 +     hybridSort(arr, left, mid);
73 +     hybridSort(arr, mid + 1, right);
74 +     // merge then back
75 +     merge(arr, left, mid, right);
76 + }
```

Data Generated

results_ci.csv		+10	0	0	0	0	0	...
...	@@ -0,0 +1,10 @@							
1	+ Input,Comparisons							
2	+ 1000,13348							
3	+ 5000,67069							
4	+ 10000,143231							
5	+ 50000,806743							
6	+ 100000,1862128							
7	+ 500000,11094923							
8	+ 1000000,23192610							
9	+ 5000000,116154701							
10	+ 10000000,242325503							

results_cii.csv		+12	0	0	0	0	0	0	...
...	@@ -0,0 +1,12 @@								
1	+ Input,Comparisons								
2	+ 5,18727236								
3	+ 10,19072873								
4	+ 15,19887197								
5	+ 20,20220967								
6	+ 25,20222346								
7	+ 30,21601105								
8	+ 35,23185773								
9	+ 40,23185788								
10	+ 50,23180124								
11	+ 75,29089533								
12	+ 100,29906352								

Theoretical Time Complexity

Merge Sort recurrence algorithm:

$$T(n) = 2T(n/2) + kn = 2(2T(n/4) + kn) + kn$$

$$= 4T(n/4) + 3kn = 4(2T(n/8) + kn) + 3kn$$

$$= 8T(n/8) + 7kn = \dots$$

$$= 2^a T(n/2^a) + a * kn$$

$$\text{At } T(1), n = 2^a \Rightarrow a = \log n$$

$$T(n) = n * T(1) + kn \log n$$

$$= O(n \log n)$$

Since recursion stops at S ,

Recursion stops at $T(S)$

$$n/2^a = S \Rightarrow a = \log(n/S)$$

$$\text{Merge cost} = O(n \log(n/S))$$

Theoretical Time Complexity

Insertion Sort Time Complexity: $O(n^2)$ (n subarrays * size n)

Since there are n/S subarrays of size S left after merge sort,

Time Complexity = $O(n/S * S^2) = O(nS)$

Overall Time Complexity: $T(n) = O(n \log(n/S)) + O(nS)$

For a fixed n , vary S : $T(S) = O(n \log(n/S)) + O(nS) = O(n \log n - n \log S + nS)$

Since n is fixed, $T(S) \propto -n \log S + nS$

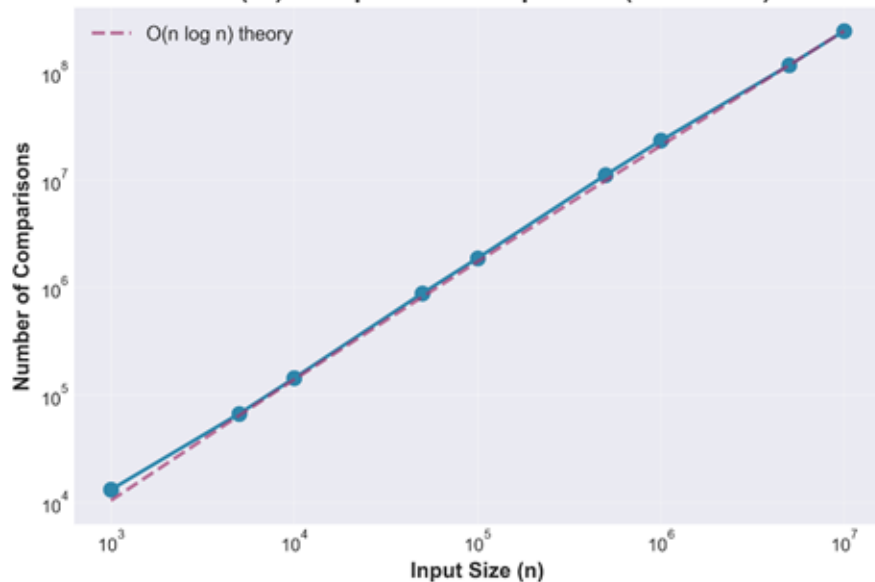
→ S too small, merge sort time becomes significant

→ S too large, insertion sort time becomes significant

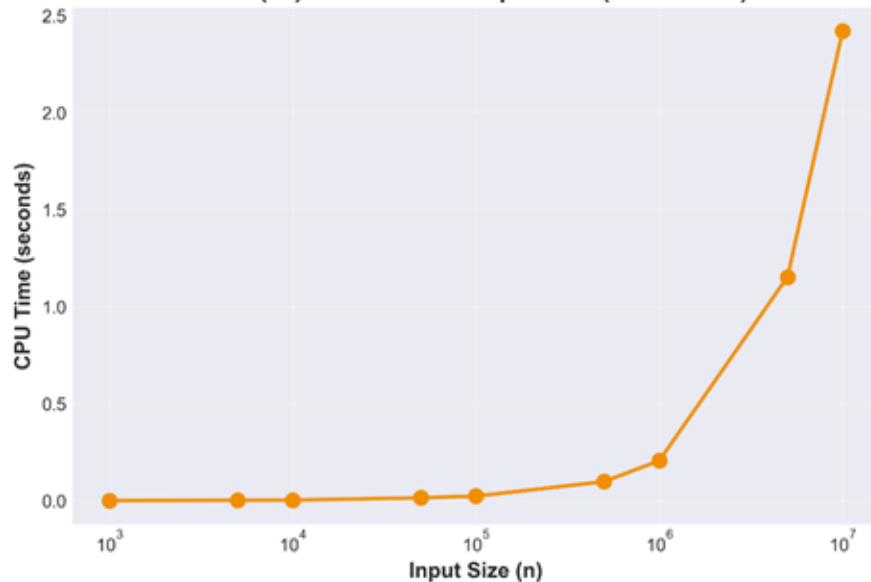
◆ Thus, S must be optimised (value around 10-50)

Empirical Time Analysis (ci)

Part (c.i): Comparisons vs Input Size (Fixed S=32)



Part (c.i): CPU Time vs Input Size (Fixed S=32)



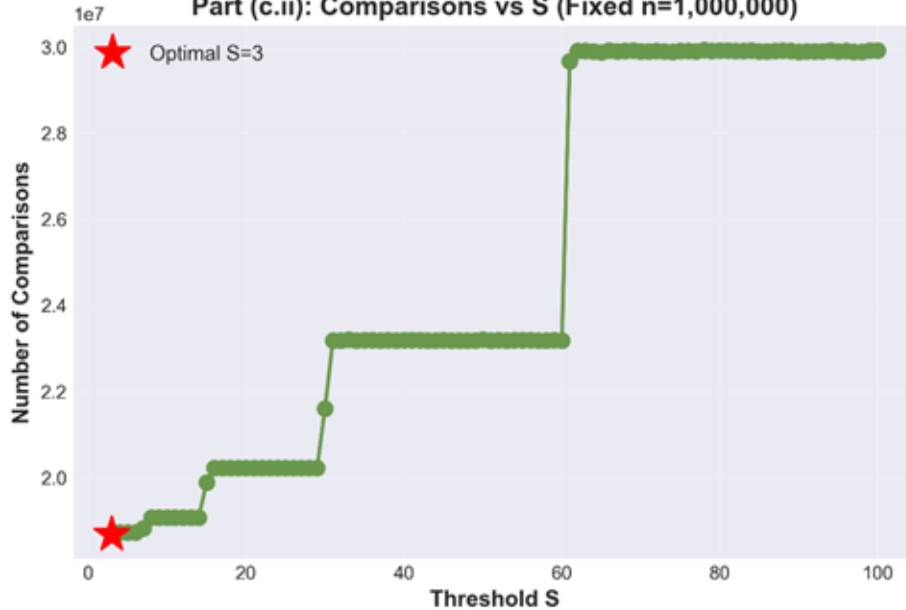
S fixed, n varies:

S = 32

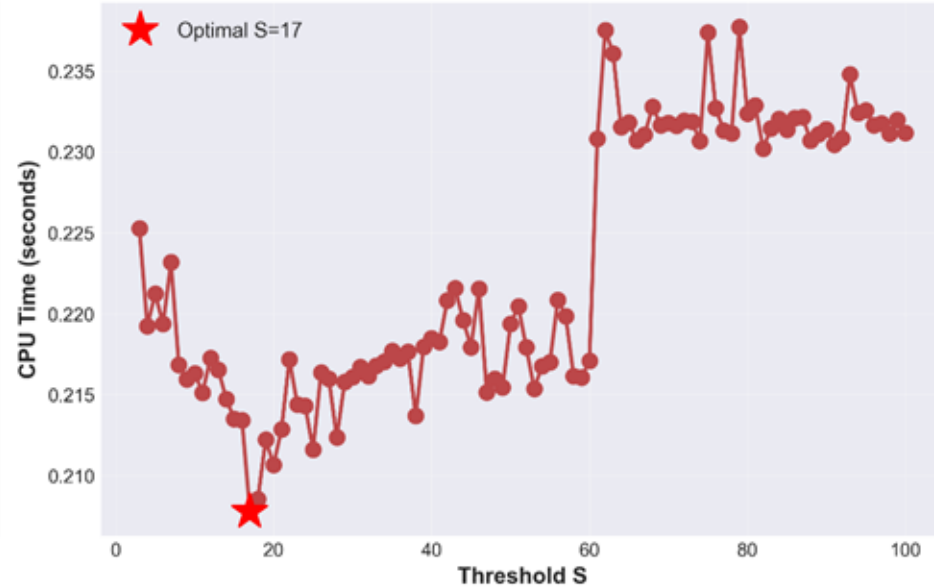
n = [1000, 5000, 10000, 50000, 100000, 500000, 1000000, 5000000, 10000000]

Empirical Time Analysis (cii)

Part (c.ii): Comparisons vs S (Fixed $n=1,000,000$)



Part (c.ii): CPU Time vs S (Fixed $n=1,000,000$)



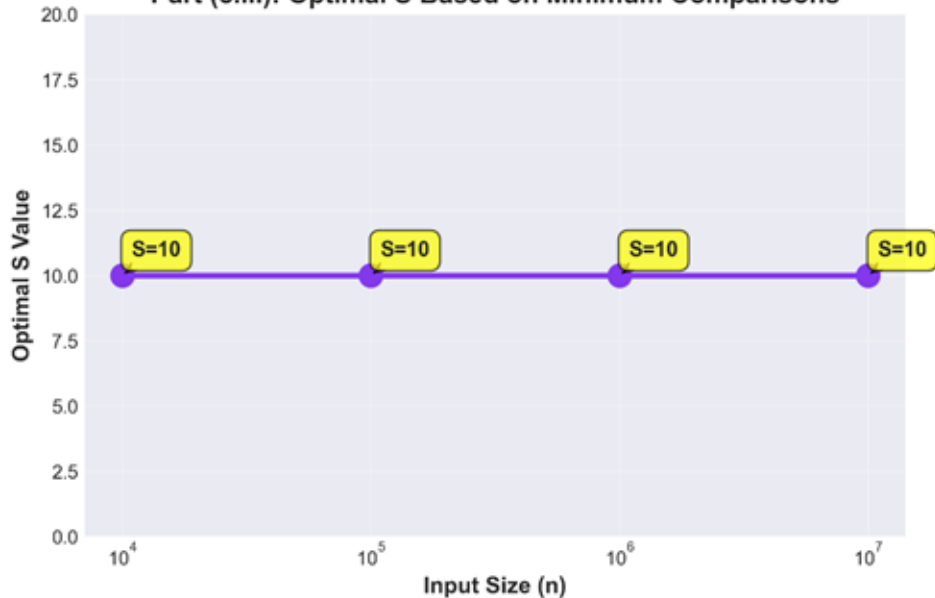
S varies, n fixed:

S = from 3 to 100 in increments of 1

n = 1,000,000

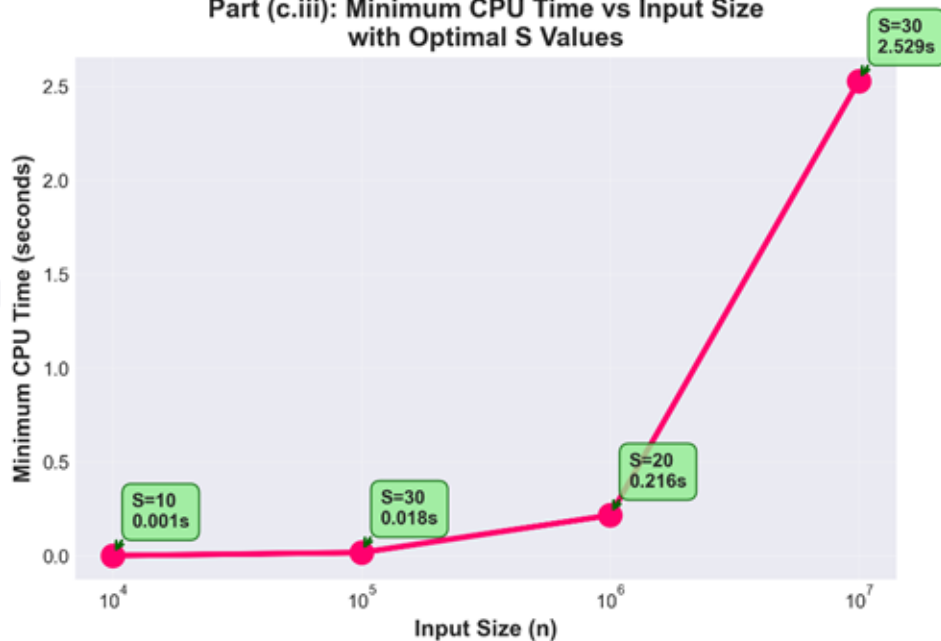
Find Empirical Optimal S (ciii)

Part (c.iii): Optimal S Based on Minimum Comparisons



Find optimal S from the values:
S = from 3 to 100 in increments of 1
n = 10k, 100k, 1M, 10M

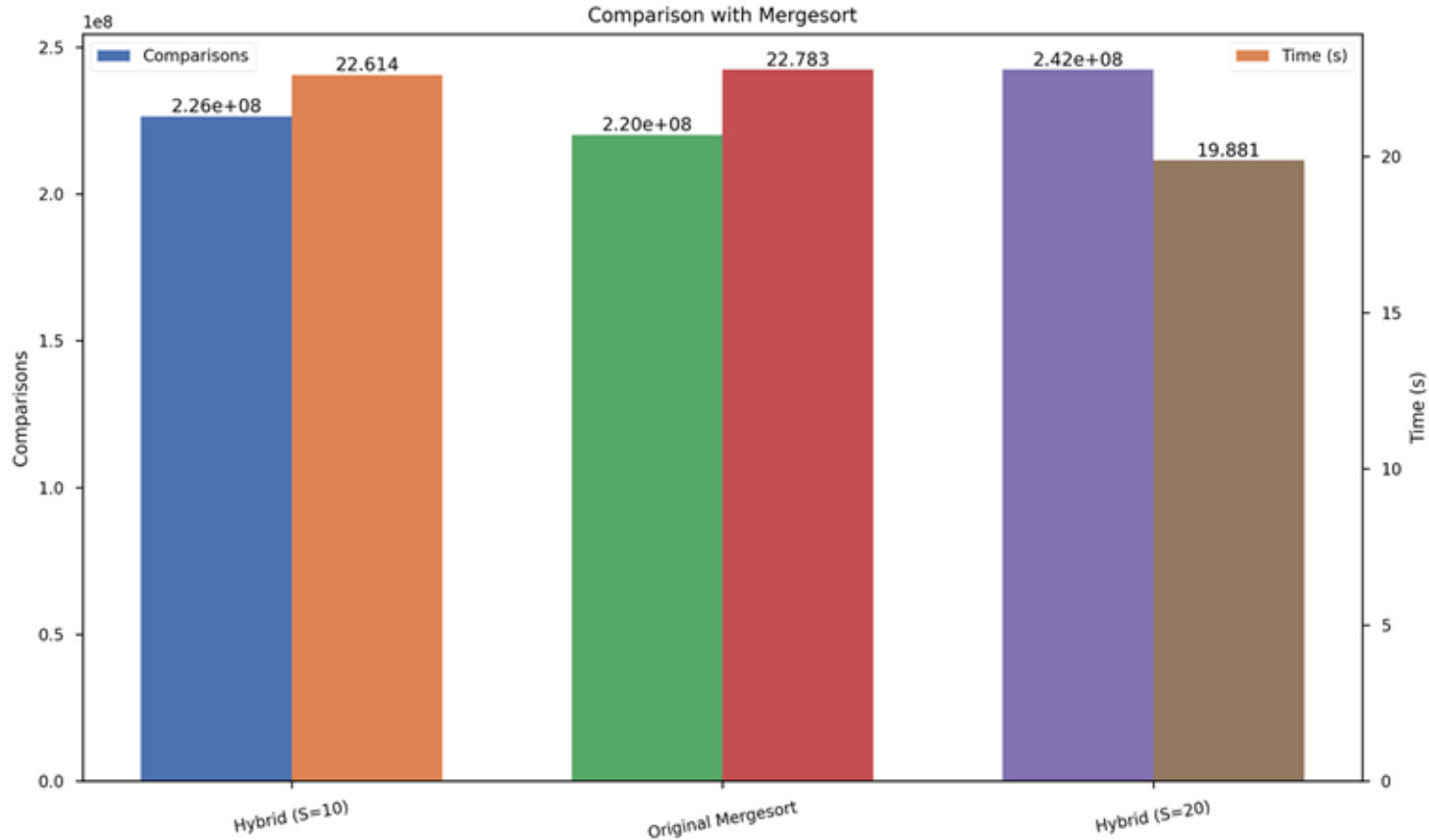
Part (c.iii): Minimum CPU Time vs Input Size with Optimal S Values



Based on Comparisons:
->Optimal S is 10

Based on CPU time:
->Optimal S is 30

Comparison Between original and hybrid Merge Sort



Algorithm	Threshold (S)	Key Comparisons	CPU Time (s)
Original Merge	0	220,101,207	22.78s
Hybrid Sort	10	226,413,222	22.61s
Hybrid Sort	20	242,308,699	19.88s

Theoretical Optimal (S=10): Focuses on minimizing the raw number of comparisons. Moving from S=0 to S=10 only slightly increases comparisons but already starts saving time.

Empirical Optimal (S=20): Even though comparisons increase to ~242M, the CPU time drops significantly to **19.88s** (a **12.7% speedup** over Original Merge)

Thank You